ARELORIA / OUROBOROS ENTWICKLER-MANUSKRIPT-MANIFEST

Konzept, Architektur, Repositoriumsstruktur, aktueller Implementierungsstand und Leitfaden für spätere Entwicklerinnen, Entwickler und Agenten

Analysierte Quelle	Wasd-main (2).zip (Repo-Snapshot)
Angefragte Remote-Quelle	https://github.com/thosu87-svg/Wasd
Dokumentdatum	17.03.2026
Status	Entwickler- und Handoff-Manuskript, kein Produktions-Freigabezertifikat

Hinweis zur Quellbasis: Ein direkter Live-Abruf der GitHub-Repo-Seite war in dieser Umgebung nicht zuverlässig möglich. Dieses Manuskript basiert deshalb auf der gelieferten ZIP-Momentaufnahme des Repositories und den darin enthaltenen Dokumentationsdateien.

Kurzfazit

Das vorliegende Repository ist kein leeres MMO-Konzept, sondern ein breit angelegter Entwicklungs-Baukasten mit klaren Leitdokumenten, einer aktiven Vite-Client- plus Express-Server-Architektur, einer großen Game-Data-Schicht sowie ausgeprägten Entwicklungs-, Admin- und Integrationsverzeichnissen. Der Snapshot vereint Vision, Runtime-Gerüst, Content-Pipeline, Tests, Deploy-Artefakte und Handoff-Dokumente in einem einzigen Projektbaum.

- Die konzeptionelle Leitidee ist ein persistentes, serverautoritäres Browser-MMORPG mit 64x64-Chunks, Observer-Simulation, datengetriebenem Content und prozeduraler Weltlogik.
- Die aktive Laufzeitarchitektur ist laut Quelllage Vite + Express/WebSocket; eine ältere Next.js-Struktur liegt noch im Root und ist als Legacy-Bestand markiert.
- Im Snapshot sind umfangreiche Domänenmodule, Testdateien, Content-Dateien und Validierungswerkzeuge vorhanden. Das Projekt ist damit ein ernstzunehmender Entwicklungsstand, aber kein eindeutiger „public release ready“-Stand.
- Besonders stark ist die Dokumentationsdichte: Lock Rules, Design Bible, Build Sequence, Final Audit, Authoring Guide und Manifest-Dateien bilden einen belastbaren Handoff-Rahmen für Menschen und Agenten.

Empfohlene Lesereihenfolge für neue Entwickler

- PROJECT_LOCK_RULES.md
- final-lock/FINAL_TRUTH.md
- docs/MASTER_DESIGN_BIBLE.md
- docs/BUILD_SEQUENCE_MASTER.md
- ARCHITECTURE_NOTE.md
- game-data/AUTHORING_GUIDE.md

Snapshot in Zahlen

Metrik	Wert
Gesamtdateien	678
Serverdateien	339
Clientdateien	89
Dokumentationsdateien	84
Game-Data-Dateien	56
Integrationsdateien	20
Admin-/GM-Dateien	10
GitHub-Workflows	3
TypeScript-Tests	40

Interpretation

Die Dateizahlen zeigen ein ungewöhnlich breites Projektprofil: Server, Client, Dokumentation, Content-Daten, Integrationen und Admin-Werkzeuge sind bereits als getrennte Verantwortungsbereiche angelegt. Besonders auffällig sind die 40 TypeScript-Testdateien im Server-Bereich, was auf einen wachsenden automatisierbaren Qualitätsrahmen hindeutet.

Konzeptkern und unverrückbare Regeln

Die Quell- und Dokumentationslage beschreibt Areloria/Ouroboros als emergentes, persistentes Browser-MMORPG. Die Vision ist nicht eine einfache Demo-App, sondern eine Welt mit Beobachter-Simulation, dynamischen Zivilisationen, NPC-Herkunft und -Erinnerung sowie wirtschaftlichen und politischen Rückkopplungen.

Konzeptachse	Beobachtung im Snapshot
Fünf Axiome	Informationsfeld, Emergenz, Persistenz, Ouroboros-Zyklus, Beobachter
Weltmodell	Prozedurale Welt, 64x64-Chunks, Biome, Ruinen, Dungeons
NPC-Doktrin	Persönlichkeit, Traits/DNA, Genealogie, Memory, Kultur/Religion
Gesellschaft	Guild -> Village -> City -> Kingdom -> Nation; Politik und Diplomatie
Spielsystem	klassenloses Skillsystem, Loot, Crafting, Handel, Magie
Metasysteme	Brain/Oracle/Dudenregister, Matrix-Energie, Weltwunder, GM-Editor, GLB-Pipeline

Lock Rules

- Das Projekt soll nicht in eine schlichte Demo-App oder Singleplayer-Sandbox zurückgebaut werden.
- 64x64-Chunks und Observer-Simulation sind Pflicht und gehören zum architektonischen Kern.
- Der Server bleibt autoritativ; der Client ist Darstellung und Eingabeoberfläche.
- NPC-Persönlichkeit, Memory, Genealogie, Politik sowie Matrix-Energie und Brain-Schicht bleiben Leitplanken.
- GM-World-Editor und GLB-Asset-Pipeline sind als dauerhafte Projektziele verankert.

Quellbasis und Prüfmethodik

Quelle	Einsatz im Manuskript
ZIP-Snapshot des Repositories	Primäre technische Quelle für Struktur, Code,

	Daten, Tests und Build-Skripte
Root- und Docs-Markdown	Leitbild, Lock Rules, Build- und Handoff-Dokumente
package.json / Workflows	Aktive Laufzeitpfade, Build-Kommandos, Deploy-Hinweise
Game-Data JSON	Aktueller authored Content: NPCs, Quests, Dialoge, Items, Spawns, Manifest
Remote-GitHub-URL	Vom Nutzer gefordert; in dieser Umgebung nicht live abrufbar, daher nur referenziert

Was dieses Dokument leistet

- Es verdichtet Konzeption, Struktur und den beobachtbaren Implementierungsstand in einem Entwickler-Manuskript.
- Es trennt sauber zwischen dokumentierter Zielarchitektur und im Snapshot klar beobachtbarer Umsetzung.
- Es dient als Handoff-Leitfaden für Menschen, No-Code-Agenten und weitere Repo-Betreuer.

Was dieses Dokument nicht behauptet

- Kein vollwertiger Produktions-Launch-Nachweis.
- Keine vollständige Laufzeit-Zertifizierung aller Module im Snapshot.
- Keine Live-Verifikation der externen GitHub-Remote-Seite jenseits des bereitgestellten Repo-Snapshots.

Aktive Runtime-Architektur

Die Root-Konfiguration startet den Server im Development-Modus über `npm run dev --prefix server`, baut den Client per Vite und den Server per TypeScript und startet produktionsnah `node server/dist/index.js`. Die Dokumentation benennt Vite Client + Express/WebSocket als aktive Laufzeit; die Next.js-Dateien im Root gelten als Legacy-Struktur und werden derzeit nicht als primäre Runtime verwendet.

Ebene	Beobachtete Rolle
Root	Orchestriert Build/Start, hält Legacy-Next-Dateien und übergreifende Dependencies
server/	Express-, WebSocket-, Game-Loop-, Persistence- und Modulschicht
client/	Vite-/Three.js-Frontend, HUD, Renderer, WebSocket-Client, Interaktionslogik
game-data/	Autoren- und Runtime-Daten: NPCs, Quests, Dialoge, Items, Spawns, Manifest
integrations/	Firebase-, AWS-, GitHub- und Gateway-bezogene

	Integrationshinweise/Adapter
admin-tools/	GM-/World-Editor- und Recovery-nahe Hilfsmittel

Wichtiger Architektursachverhalt

Im Snapshot existiert sowohl ein Root-Next.js-Bestand als auch eine dokumentierte, offenbar aktuell bevorzugte Vite/Express-Architektur. Für spätere Entwickler bedeutet das: Vor jeder größeren Umstrukturierung muss klar entschieden werden, ob der Legacy-Root endgültig entfernt, konserviert oder für andere Zwecke weitergeführt wird.

```
Root-Skripte

  npm install
  npm run build
  npm run start

Server-spezifisch

  cd server
  npm run validate
  npm run build
  npm run dev

Client-spezifisch

  cd client
  npm run build
  npm run dev
```

Repositoriumslandkarte

Pfad	Zweck für Entwickler
docs/	Leitdokumente, Vision, Build-Reihenfolge, Konsistenz- und Finalisierungsnotizen
final-lock/	Unterschriebene Kernwahrheiten und Anti-Simplifizierungs-Leitplanken
agent/	Prompts und Fail-Safe-Regeln für Agentenbetrieb
server/src/modules/	Domänenmodule in breiter fachlicher Aufteilung
server/src/core/	WorldTick, Persistence, Cache, zentrale Laufzeitkoordination
server/src/networking/	WebSocket- und Session-/Transportlogik

client/src/engine/	Three.js-Szene, Renderer, Welt- und Anzeige-Mechanik
client/src/ui/	HUD, Panels, Tooltips, Quest-/Inventory-/Dialog-UI
game-data/	Autorendaten und Content-Manifest
integrations/	Cloud-, Deployment- und Service-Adapterschicht
admin-tools/	GM- und Recovery-Werkzeuge
.github/workflows/	Azure-orientierte Build/Deploy-Workflows

Server-Domänenfamilien (gekürzt)

Die Modulstruktur deckt bereits viele Fachbereiche ab: world, observer, npc, quest, inventory, combat, crafting, guild, social, faction, politics, history, oracle, reputation, weather, growth, housing, market, loot, payment, monitoring, telemetry, brain, AI und world-editor. Das ist architektonisch ambitioniert; deshalb sollte jede Erweiterung über die vorhandenen Domänengrenzen laufen und nicht als neue Querlogik daneben gebaut werden.

Content-Pipeline und Autorenfluss

Content-Bereich	Menge im Snapshot	Bemerkung
NPC-Definitionen	14	u. a. Questgeber, Gegner, Händler, Hermit, Apprentice, Shopkeeper
Quest-Definitionen	12	im Snapshot überwiegend Single-Step-Quests mit Prerequisites
Dialog-Sets	11	datengetrieben verknüpft über dialogueId
Items	26	Waffen, Armor, Consumables und Misc-Items
Spawn-Platzierungen	18	aktuell in einer Region gesammelt
Content-Packs im Manifest	1	derzeit ein Sammelpack „Core Content“

Beobachtete Eigenschaften der Content-Schicht

- NPCs referenzieren Dialoge und QuestHooks über Daten statt über harte Laufzeitverdrahtung.
- Quests liegen im Snapshot datengetrieben vor; drei Quests nutzen prerequisiteQuestIds.
- Items sind zentral in `game-data/items/items.json` beschrieben und dienen als Quelle für Werte und Tooltips.

- Ein automatisiertes `content-manifest.json` fasst Content-Packs, IDs und Validierungsstatus zusammen.
- Der Authoring Guide beschreibt, wie neue NPCs, Dialoge, Quests und Spawns hinzugefügt werden.

Validator-Pipeline

Im Server-Build ist eine Content-Validierung vorgeschaltet. Der Validator prüft doppelte IDs, kaputte NPC-/Quest-/Item-/Spawn-Referenzen und die Erreichbarkeit von Dialogknoten. Damit wird authored Content schon vor dem eigentlichen TypeScript-Build abgefangen.

```
Validator (Server)

  cd server
  npm run validate

Automatisch vor Build

  npm run build
```

Beobachtbarer Implementierungsstand

Bereich	Einschätzung	Kommentar
Root- und Leitdokumente	Stark	Sehr hohe Dichte an Lock-, Audit-, Manifest- und Handoff-Dateien
Runtime-Grundgerüst	Stark	Server/Client/Networking/ Persistence/Validation sind klar angelegt
Data-driven Content	Stark	NPCs, Dialoge, Quests, Items, Spawns und Manifest sind authored
Questmodell	Solide, aber aktuell simpel	Im Snapshot vor allem Single-Step-Quests mit Prerequisites; keine sichtbaren Multi-Step-Daten
Inventory/Equipment/Loot	Vorhanden	Dateien und HUD-/Client-/Server-Pfade deuten klar auf implementierten Slice
NPC-/World-Meta-Systeme	Breit angelegt, Reife uneinheitlich	Viele Module und Konzepte; nicht alles ist im Snapshot als produktionsreif belegbar
Admin/GM und Asset-Pipeline	Angelegt	Tooling, Registries und Leitdokumente sind vorhanden, endgültige Tiefe bleibt zu

		verifizieren
Deploy/Integrationen	Teilweise	Firebase/AWS/Azure-Hinweise und Workflows existieren, aber Zielbild ist noch fragmentiert

Besonders greifbare Code-Indizien

- WebSocket-Login und automatischer Character-Name-/Recovery-Fluss im Clientcode.
- WorldTick als zentraler Koordinator für Connect, Tick, Loot-Entitäten, Equip/Unequip, Cooldowns und Persistenz-Hooks.
- ItemRegistry und InventorySystem als wiederkehrender Pfad für Reward-, Drop- und Equipment-Logik.
- HUD- und Panel-Code für Tooltip, Inventory, Stats, Questanzeige und Interaktionshilfen.
- Test-Suite mit 40 TypeScript-Tests plus Markdown-Testplänen im Serverbereich.

Qualitätssicherung, Tests und Deployment

Aspekt	Snapshot-Befund
Automatisierte Tests	40 TS-Testdateien plus ergänzende Test-Markdown-Dateien im Server-Bereich
Content-Validierung	Vor Server-Build integriert
GitHub Actions	Drei Azure-orientierte Workflow-Dateien im Repo
CircleCI	Im Snapshot nicht vorhanden
Manifest-/Authoring-Schutz	content-manifest.json + validateContent.ts vorhanden

Deploy-Situation

Das Projekt zeigt mehrere Zielwelten gleichzeitig: Firebase/Firebase Studio, AWS/RDS/ElastiCache, Azure Web App sowie eigene VPS-/PM2-Deploy-Hinweise. Für neue Entwickler ist das ein Warnsignal: Vor einem echten Release sollte eine einzige primäre Zielplattform festgelegt werden, sonst droht eine zersplitterte Ops-Landschaft.

Leitfaden für spätere Entwickler

Empfohlene Onboarding-Reihenfolge

- Zuerst Lock Rules, Final Truth, Design Bible und Build Sequence lesen.
- Dann die aktive Runtime (Vite + Express/WebSocket) und die Legacy-Root-Struktur voneinander trennen.
- Vor jeder Feature-Arbeit den Content-Validator und die vorhandenen Tests kennenlernen.

- Neue Inhalte möglichst ausschließlich über `game-data/` und die Manifest-/Validierungskette hinzufügen.
- Kernbereiche wie WorldTick, Chunk/Observer, Persistenz, Item Registry und Quest-/Inventory-Fluss nur mit kleinstmöglichen Änderungen anfassen.

Sinnvolle erste Arbeitskommandos

```
1) Root-Abhängigkeiten installieren
   npm install

2) Content validieren
   cd server && npm run validate

3) Client und Server bauen
   cd .. && npm run build

4) Server starten
   npm run start
```

Was man nicht leichtfertig umbauen sollte

- WorldTick und alle Pfade, die Tick-, Persistenz- und Interaktionslogik bündeln.
- Chunk- und Observer-Systeme sowie Serverautorisierung.
- Item Registry, Inventory-/Equipment- und Hydration-/Persistenzpfade.
- Datengetriebene Content-Pipeline samt Manifest- und Validierungslogik.
- Die Lock Rules selbst.

Wichtige Risiken und Entwicklungsnähte

Risiko / Nahtstelle	Warum relevant
Legacy-Next + aktive Vite/Express-Struktur	Kann neue Entwickler verwirren und Build-/Deploy-Pfade vermischen
Dokumentationstiefe vs. Runtime-Reife	Einige Dokumentschichten entwerfen mehr Systemtiefe als der Snapshot sicher belegt
Ops-Zielbild fragmentiert	Firebase, AWS, Azure und VPS tauchen parallel auf
Questmodell im Snapshot eher simpel	Mehrschrittige authored Quests sind in den Daten dieses Snapshots nicht sichtbar
Auth-/Identitätslage wirkt hybrid	README spricht Firebase, Clientcode zeigt zugleich einen leichtgewichtigen Name-/Recovery-Flow

Release-Reife des Snapshots

Der Snapshot ist stark genug für interne Weiterentwicklung, Agenten-Handoffs und strukturierte Implementierung. Er sollte jedoch nicht als automatisch öffentliche, bugfreie Release-Fassung gelesen werden. Dafür wären ein konsolidiertes Ops-Ziel, eine eindeutige Auth-Strategie, strengere Laufzeitverifikation und eine klare Priorisierung zwischen dokumentierter Vision und tatsächlich aktivem Produktionspfad nötig.

Empfohlene nächste Arbeitspakete

- Runtime und Docs synchronisieren: Aktiv genutzte Pfade, Zielplattform und Auth-Story eindeutig machen.
- Content weiter vertiefen, aber generisch halten: authored Quests, Dialoge und Spawns bevorzugen, Runtime-Kern nicht verbiegen.
- Validator und Tests ausbauen, bevor große neue Systeme starten.
- CI/CD standardisieren: eine Zielplattform, ein klarer Quality Gate, keine Deploy-Sprawl-Logik.
- Legacy-Bestand nur dann löschen, wenn Build- und Startpfade nachweislich stabil konsolidiert wurden.

Kurzfristiger Entwicklerfokus

Zeithorizont	Empfohlener Fokus
Kurzfristig	Build-/Startpfade glätten, Content-Validator nutzen, authored Content sicher erweitern
Mittelfristig	Quest-/UI-/Combat-Slice stabilisieren, Tests erweitern, Ops-Zielbild vereinheitlichen
Später	Tiefe World-/Brain-/Politics-/Matrix-Layer gegen die Lock Rules weiter hochziehen

Anhang A - Top-Level-Ordner als Schnellkarte

Ordner	Status im Snapshot
.Jules	vorhanden
.jules	vorhanden
.github	vorhanden
admin-tools	vorhanden
agent	vorhanden
app	vorhanden
client	vorhanden

deploy	vorhanden
docs	vorhanden
final-lock	vorhanden
game-data	vorhanden
hooks	vorhanden
integrations	vorhanden
lib	vorhanden
server	vorhanden
src	vorhanden
tests	vorhanden
world-assets	vorhanden

Anhang B - Inhaltsdaten (Core Content Manifest)

Manifest-Feld	Wert
Pack-Name	Core Content
NPC-IDs	npc_1, npc_2, npc_dummy, npc_3, npc_4, npc_5, npc_wolf, npc_6_hermit ...
Quest-IDs	first_steps, missing_letter, dummy_slayer, iron_scraps, scouts_report, wolf_hunt, pelt_collector, master_work ...
Dialogue-IDs	dialogue_npc_1, dialogue_npc_2, dialogue_npc_3, dialogue_npc_4, dialogue_npc_5, dialogue_wolf ...
Item-IDs	starter_sword, wooden_shield, iron_scrap, wolf_pelt, fang, alpha_claw, bone_fragment, rusted_blade ...
Validierungsstatus	passed

Anhang C - Praktische Notiz für Handoffs

Wenn dieses Dokument an spätere Entwickler oder an einen Agenten weitergegeben wird, sollte immer zusammen mit dem Snapshot selbst gearbeitet werden. Dieses Manuskript beschreibt, was im Snapshot erkennbar ist, ersetzt aber weder den Code noch die Build-/Validierungswerkzeuge. Der sicherste Einstieg bleibt: lesen, validieren, bauen, dann erst ändern.